

Параллельное программирование

Домашняя работа №3. Множество Мандельброта

Выполнил: Деревянкин Тимофей
Группа: 24-по-1
Дата: 08.10.2026

Цель и постановка задачи

Рассчитать для каждого пикселя принадлежность соответствующей точки комплексной плоскости множеству Мандельброта с использованием нескольких потоков. Вывод изображения опционален; в реализации предусмотрены PNG и ASCII.

Алгоритм

Для центра каждого пикселя выбирается $c = c_x + i \cdot c_y$ из области $[-2.5; 1.0] \times [-1.25; 1.25]$. Итерации начинаются с $z = 0$; на каждом шаге вычисляется $z = z^2 + c$. Выход при $|z|^2 > 4$ означает, что точка вне множества. Если выход не обнаружен за 1000 итераций, сохраняется -1. Это конечное приближение принадлежности, особенно около границы.

Организация потоков

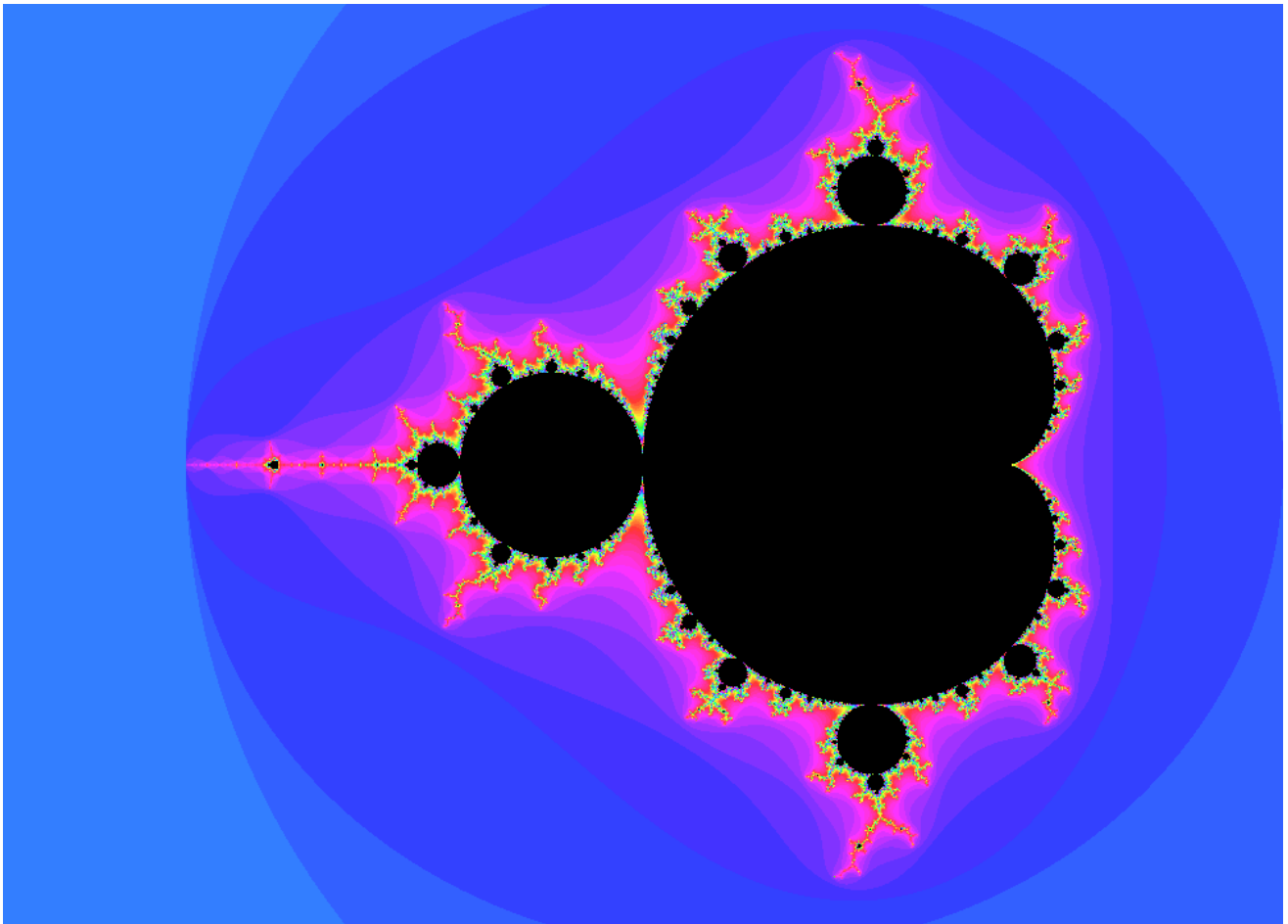
FixedThreadPool содержит шесть работников. Каждый берёт индекс следующей строки через AtomicInteger, рассчитывает все её пиксели и берёт новую строку. Так занятые строки не дублируются, а свободный поток может продолжить работу, не ожидая завершения постоянного блока. Записи идут в разные строки массива, поэтому пиксельные блокировки не требуются. Future.get() ожидает работников и обеспечивает видимость результатов.

Результат пробного запуска

Параметр	Результат
Размер / предел итераций	1000 × 720 / 1000
Параллельный расчёт, 6 работников	79.102 мс
Последовательный расчёт	413.857 мс
Ускорение в этом запуске	5.23
Не вышли за радиус 2	124278 из 720000 пикселей
Сверка каждого пикселя	Полное совпадение

Среда: Intel Core i5-11400F, 6 физических и 12 логических ядер, Windows 11, Temurin 21.0.12.1. Перед расчётом выполнен короткий прогрев. Запись PNG исключена из замера. Один запуск не является статистической оценкой ускорения.

Рассчитанное изображение



Чёрные пиксели не вышли за радиус 2 за установленное число шагов; цвет снаружи зависит от количества итераций до выхода.

Проверка корректности

Проверены точки 0, -1, -2 (не выходят), 1 и 2 (выходят). Отдельно проверен выход ровно на последней итерации. Для сеток 101×43, 7×3 и 1×1 при 1, 2, 6 и 8 потоках каждый результат сравнивается с последовательной версией. Включён случай, когда потоков больше, чем строк.

Запуск

```
java -jar Mandelbrot.jar --verify --png mandelbrot.png  
java -jar Mandelbrot.jar --check
```

Вывод

Каждый пиксель рассчитан параллельно, результат совпадает с последовательным алгоритмом. Динамическое распределение строк уменьшает простой при неодинаковой стоимости расчёта разных частей изображения. Для очень маленькой сетки расходы на запуск потоков могут оказаться больше выигрыша от параллельности.